AI2 · Aplicaciones con LLMs

Prompts de producción · Salidas estructuradas · Ferramentas

AI Surfer · N3-AI Engineering

Mapa del módulo

Pieza	Pregunta que responde
System prompt	¿Qué reglas hace que el modelo no improvise?
Salida estructurada	¿Cómo consumo la respuesta desde código?
Ferramentas	¿Cómo conecta el modelo a mis datos?
Streaming	¿Por qué el usuario percibe velocidad?
Contexto	¿Qué llevo en la ventana y qué resumo?
Caché	¿Cómo bajo el coste de un asistente?

System prompt que aguanta

1. Rol y dominio
2. Reglas **accionables** (verificables)
3. Frase de escape exacta
4. Formato de la respuesta
5. Tono (1 línea)

Regla vaga ("sé útil") = depuración imposible.

Salida estructurada (Pydantic)

- El schema viaja en el prompt — el modelo ve tipos
- `model_validate_json` valida o lanza error **nombrado**
- Si falla: re-llamada con el error en el prompt

> Anti-patrón: `json.loads` y rezar.

Ferramentas (function calling)

1. Declaras nombre + descripción + schema
2. El modelo emite `tool_use` (decide **él**)
3. Tu código ejecuta y devuelve `tool_result`
4. El modelo responde con el dato real

Límite de vueltas: 3. Errores: devuélvelos en el `tool_result`.

Streaming

- Los tokens llegan por bloques → píntalos a medida
- El tiempo **total** no cambia; la **percepción** sí
- No streamar JSON a validar: esperar y renderizar

Contexto y caché

- Contexto finito: truncar / **resumir** / RAG (AI3)
- Caché de prompt: la parte **estable** (system + docs) se paga muy reducida
- El historial y la pregunta: al final, sin caché

Resumen

Reglas accionables → schema validado → herramientas → streaming → caché

Con esto el LLM deja de ser demo y pasa a ser producto. Lo que sigue: RAG y agentes (AI3).